Plataformas de Desenvolvimento

24/25

Mestrado em Informática e Sistemas

1

Jenkins

- Why Jenkins?
- What are Jenkins Jobs
- What is Jenkins DSL
- What is Jenkins Pipeline
- Email Integration
- Slack Integration

2

Jenkins

- Jenkins is an Open Souce CI and CD tool written in Java.
- Jenkins was previsouly know has Hudson.
- Main goal to build and deliver software.
- There are other tools on the market. Jenkins is open source and very popular.

3

Jenkins Alternatives

- Self-hosted
  - Drone CI
  - TeamCity
- As a service
  - CircleCI
  - CodeShip
  - SemahoreCI
  - GitHub Actions

4

Jenkins

- Several plugins available:
  - Email integration
  - Dashboards
  - Slack
  - Docker
  - …

5

Jenkins Benefits

- With the plugins available it is possible to notify the developer team when some issues appear.
- Quicker bug report improve bug fixing speed.
- It is possible to publish and deploy all the software tested by Jenkins.
- Using an autonomous tool like Jenkins improve speed to QA, UAT or even Production environments.

6

Jenkins

- Installation:
  - We will use docker
  - Same version for everyone
  - Easier to upgrade
  - Possible to create custom images with the jenkins baseline

- Use Cases:
  - Jenkins UI Jobs
  - DSL
  - Pipelines
  - Email/slack/…

7

Jenkins UI Jobs

- Allow to create through the UI the creation of jobs to:

- Install
  - Download and install all dependencies
- Run tests
  - Run tests and notify developers in case of failed tests
- Package the application
  - Create zip, rpms , docker image,s etc gathering all the binaries and dependencies to run the application
- Distribution (rpms, docker images , etc…)
  - Docker registry

8

Jenkins UI Jobs

- Allow to have a UI to configure jobs to input all the **build parameters**
- Disadvantages:
  - No audit log
  - No track changes
  - Isolation between developers and Jenkins administrators:
    - Developers must contact administrators to change the Job
    - Manual tasks and burocracy leads to higher developer time
  - Difficult to rollback changes (e.g. rollback to previous day version)

9

Jenkins automation – Infrastructure as code

- **Write jobs as code** and save it to the version control repositor
- Git/svn gives you the historical changes for the files and audit trail
- Easier to rollback to previous version of jobs and build instructions.
- Union between developers and Jenkins admins
  - Developers can interact with the jobs and build parameters into their own repositor (e.g. a Jenkinsfile)

10

Jenkins automation – Infrastructure as code

- **DevOps –** developers control their own builds
  - **Jenkins DSL**
  - **Jenkins Pipeline**

11

Job DSL and Jenkins Pipeline

- **Jenkins Job DSL**
  - Write code that creates and modify jenkins job automatically
- **Jenkins Pipeline (JenkinsFile)**
  - Bundle the build parameters within the project
  - Allow developers to change jenkins build parameters
  - Enable audit , backup and restore through version control repositories.

12

Jenkins Job DSL

- It is a plugin that allows to define a job programatically
- **DSL – Domain Specific Language**
- Possible to define jobs using Groovy based language
  - Similar to java
  - Allows to script your builds and pipeline

13

Jenkins Job DSL

```
job('example') {
  steps {
    shell('echo Hello World!')
  }
}
```

- Will create a new job named example
- Have 1 step
  - Shell that prints "Hello world"

14

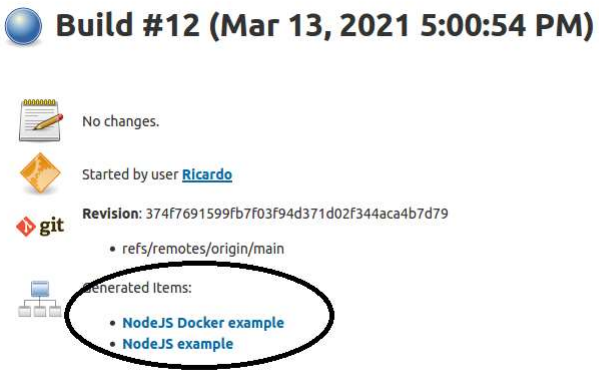
Jenkins Job DSL

```
job('NodeJS example') {
    scm {
        git('git://github.com/RicardoFilipe/docker-example.git') {  node -> // is hudson.plugins.git.GitSCM
            node / gitConfigName('DSL User')
            node / gitConfigEmail('jenkins-dsl@newtech.academy')
        }
    }
    triggers {
        scm('H/2 * * * *')
    }
    wrappers {
        nodejs('nodejs') // this is the name of the NodeJS installation in
                         // Manage Jenkins -> Configure Tools -> NodeJS Installations -> Name
    }
    steps {
        shell("npm install")
    }
}
```

- Get info from Git (configname and email necessary to access github)
- The job will run 2 times per hour
- Will get the nodejs installation
- Run the installation steps.

15

Jenkins Job DSL



- 2 new jobs created from the DSL JOB

16

Jenkins Job DSL

- https://jenkinsci.github.io/job-dsl-plugin/#path/job

- `parameters`: setting parameters for users to input when they create a new build of the job.
- `properties`: static values that are to be used within the job.
- `scm`: configuration for how to retrieve the source code from a source-control management provider like GitHub.
- `steps`: definitions for each step of the build.
- `triggers`: apart from manually creating a build, specifies in what situations the job should be run (for example, periodically like a cron job, or after some events like a push to a GitHub repository).
- `description` – `Job description`:

17

Jenkins Job DSL – Publish on dockerhub

Examples

```
job('example') {
    steps {
        dockerBuildAndPublish {
            repositoryName('example/project-a')
            tag('${BUILD_TIMESTAMP}-${GIT_REVISION,length=7}')
            registryCredentials('docker-hub')
            forcePull(false)
            createFingerprints(false)
            skipDecorate()
        }
    }
}
```

18

Pipeline vs DSL

- They both have the capability to write CI/CD code
- They are diferent in Jenkins
- Jenkins Job DSL create new jobs based on the script

19

Pipeline

- Specific job type that can be created using the UI or the Jenkin Job DSL.
- You can write the pipeline in groovy (scripting pipeline) or declarative pipeline
- Jenkins Pipeline is a job type , handling the build/test/staging/production of the project.

20

Pipeline

- Everytime a developer make a change in the repositor a build might start in jenkins
- Pull – Jenkins pulls the version each X minutes
- Push – The version control (e.g. git) will send a notification to Jenkins.
  - Github plugin to push notifications

21

Pipeline - scripted (groovy)

```
node {

   def mvnHome

   stage('Preparation') {
      git 'https://github.com/ward/java-demo.git'
      // Get the Maven tool.
      // ** NOTE: This 'M3' Maven tool must be configured
      // **       in the global configuration.
      mvnHome = tool 'M3'
   }

   stage('Build') {
      // Run the maven build
      if (isUnix()) {
         sh "'${mvnHome}/bin/mvn' -Dmaven.test.failure.ignore
clean package"
      } else {
         bat(/"${mvnHome}\bin\mvn" -Dmaven.test.failure.ignore
clean package/)
      }
   }

   stage('Results') {
      junit '**/target/surefire-reports/TEST-*.xml'
      archive 'target/*.jar'
   }

}
```

- Node: influence on what jenkins worker node the job will be ran (here: any node)
- def: allows you to declare variables
- Stage: defines a building stage: build, test, or deploy
  - Conceptually distinct step
  - Used by other plugins in Jenkins later on to visualize the stages of a job
    - e.g. clean->build->test->publish

22

Pipeline - scripted (groovy) : other example

```
node {
   def commit_id
   stage('Preparation') {
     checkout scm
     sh "git rev-parse --short HEAD > .git/commit-id"
     commit_id = readFile('.git/commit-id').trim()
   }
   stage('test') {
     nodejs(nodeJSInstallationName: 'nodejs') {
       sh 'npm install --only=dev'
       sh 'npm test'
     }
   }
   stage('docker build/push') {
     docker.withRegistry('https://index.docker.io/v1/', 'dockerhub') {
       def app = docker.build("ricardofilipe/docker-nodejs-demo:${commit_id}", '.').push()
     }
   }
}
```

- 3 stages
- 1 def variable (commit_id)
- ‘nodejs’ string already available at Jenkins
- ‘dockerhub’ string already available at Jenkins

23

Pipeline - declarative

```
pipeline {
  agent { label 'node-1' }
  stages {
    stage('Source') {
      steps {
        git 'https://github.com/ricardofilipe/jenkins-tutorials.git'
      }
    }
    stage('Compile') {
      tools {
        gradle 'gradle4'
      }
      steps {
        sh 'gradle clean compileJava test'
      }
    }
  }
}
```

- Same goal as scripted pipeline
- Relatively new
- Different syntax

24

Pipeline – docker plugin

Usefull to build/publish docker images.

It is possible to run existing containers.

Usefull to create testing envronments.

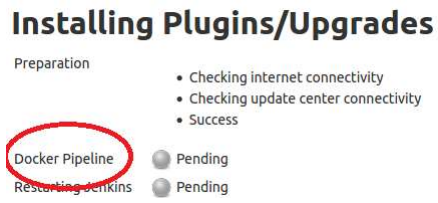
Run specific containers.

Remove the containers after the tests are made.

Usefull for multiple builds at the same time (each one will have is own container)

25

Pipeline



- First you need to install Docker Pipeline plugin to be able to publish to dockerhub

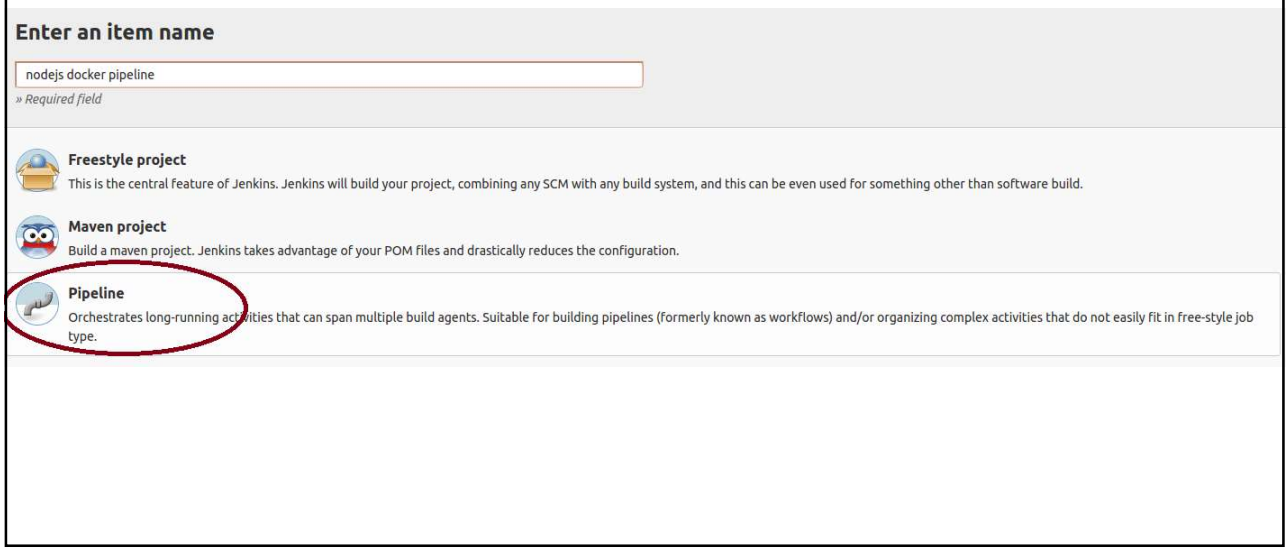
26

Pipeline – docker plugin

```
node {
   def commit_id
   stage('Preparation') {
     checkout scm
     sh "git rev-parse --short HEAD > .git/commit-id"
     commit_id = readFile('.git/commit-id').trim()
   }
   stage('test') {
     def myTestContainer = docker.image('node:4.6')
     myTestContainer.pull()
     myTestContainer.inside {
       sh 'npm install --only=dev'
       sh 'npm test'
     }
   }
   stage('test with a DB') {
     def mysql = docker.image('mysql').run("-e MYSQL_ALLOW_EMPTY_PASSWORD=yes")
     def myTestContainer = docker.image('node:4.6')
     myTestContainer.pull()
     myTestContainer.inside("--link ${mysql.id}:mysql") { // using linking, mysql will be available at host: mysql, port: 3306
          sh 'npm install --only=dev'
          sh 'npm test'
     }
     mysql.stop()
   }
   stage('docker build/push') {
     docker.withRegistry('https://index.docker.io/v1/', 'dockerhub') {
       def app = docker.build("ricardofilipe/docker-nodejs-demo:${commit_id}", '.').push()
     }
   }
}
```

27

Pipeline – create new pipeline job



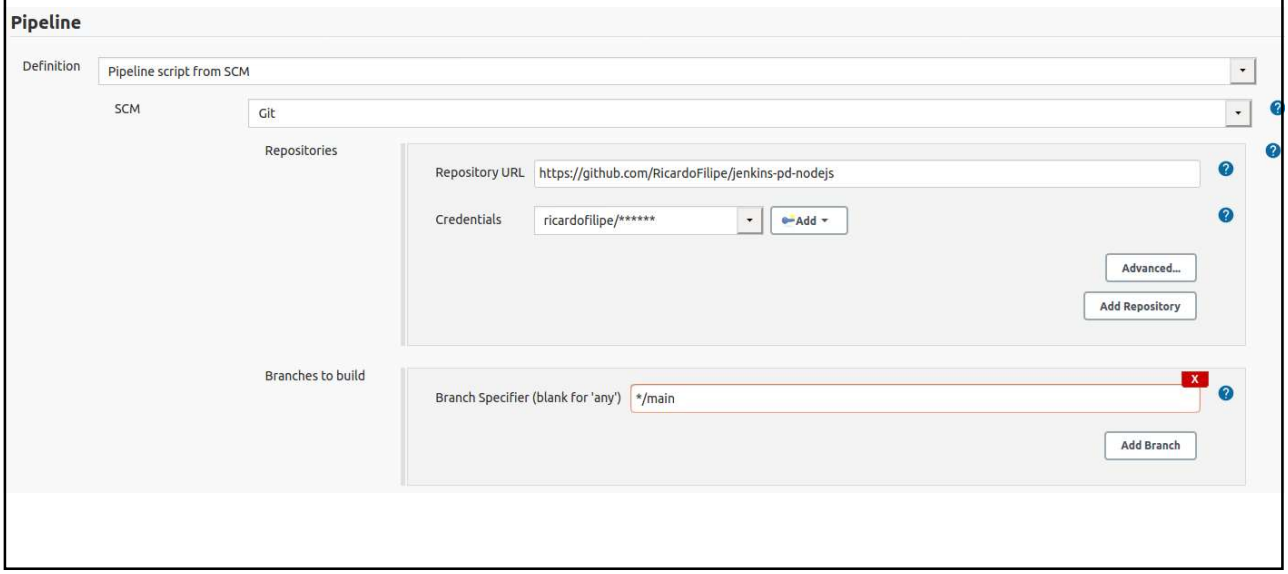
28

Pipeline – create new pipeline job



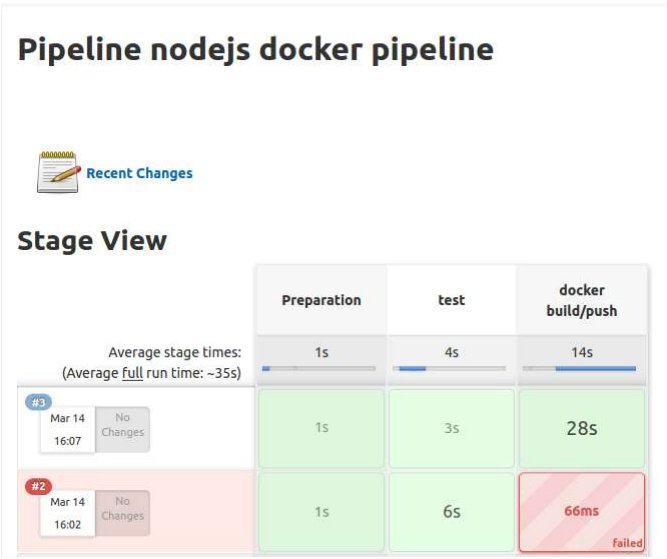
29

Pipeline – create new pipeline job



30

Pipeline – create new pipeline job



- Visualize each step
- Understand what went wrong

31

Pipeline- other points

Add timestamp to your build, to understand where your build consumes time

```
21:51:15  Started by user anonymous
21:51:15  Building on master
21:51:17  Finished: SUCCESS
```

```
timestamps {
  // do your job
}
```

```
timestamps {

    stage('Checkout')
        checkout scm:
```

32

Pipeline- other points

Create new parameters

Configure build Rotation

Pipeline triggers

```
properties([
        parameters(
            [
                [ $class: 'TextParameterDefinition', name: 'repoUrl', defaultValue: 'https://github.com/RicardoFilipe/jenkins-pd-nodejs' ],
                [ $class: 'BooleanParameterDefinition', name: 'skipDockerFiles', defaultValue: 'True' ]
            ]
        ),
        buildDiscarder(
                logRotator(
                        artifactDaysToKeepStr: '',
                        artifactNumToKeepStr: '',
                        daysToKeepStr: '30',
                        numToKeepStr: '15'
                )
        ),
        disableConcurrentBuilds(),
        pipelineTriggers([
                pollSCM(ignorePostCommitHooks: true, scmpoll_spec: 'H 12,17,20 * * * ')])
])
```

33

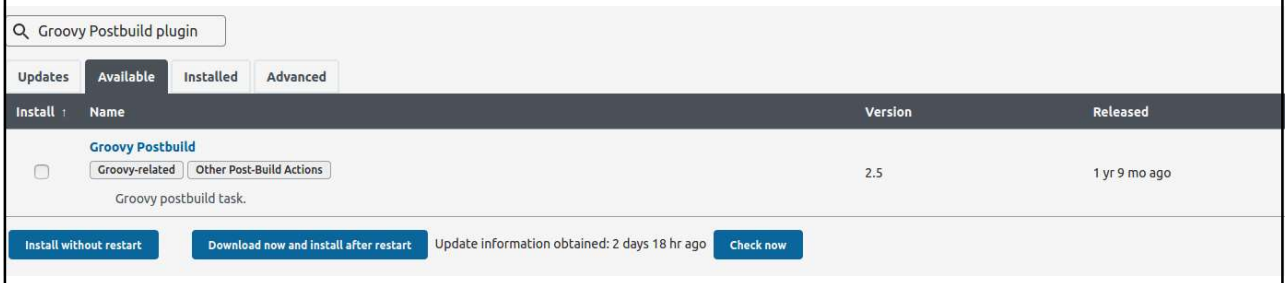
Pipeline- other points



34

Pipeline- manager

Usefull to customize your build reports



https://plugins.jenkins.io/groovy-postbuild/

35

Pipeline- manager

```
if (manager.logContains(".*properties.*")) {
            manager.addWarningBadge("Thou shalt not use deprecated methods.")
            manager.createSummary("warning.gif").appendText("<h1>You have been warned!</h1>", false, false, false, "red")
            manager.buildUnstable()

}
```





https://plugins.jenkins.io/groovy-postbuild/

36

Pipeline- try catchs and email/slack integration

```
try {

    stage('Checkout') {
        checkout scm: (...)
    }

    // Running the build container
    stage('Deploy') {
            (...)
        }
    }

} catch (e) {
    echo 'Build failed failed: ' + e
    currentBuild.result = 'FAILED'

    script {
        if (manager.logContains(".*ERROR.*")) {
            manager.buildUnstable()
        }
    }

    throw e
} finally {
    currentResult = currentBuild.result ?: 'SUCCESS' // Elvis operator

    def previousResult = currentBuild.previousBuild?.result
    // mark build as failed

    // set variables
    def subject = "${env.JOB_NAME} - Build #${env.BUILD_NUMBER} ${currentBuild.result}"
    def content = '${JELLY_SCRIPT,template="html"}'

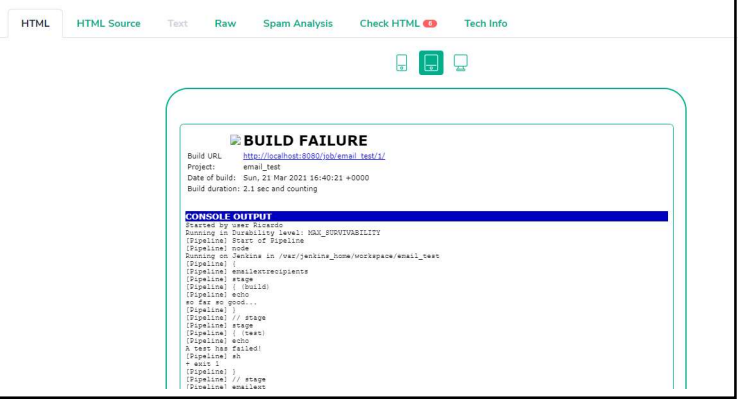
    //send email
    if (previousResult != null && previousResult != currentResult) {
        if(to != null && !to.isEmpty()) {
          emailext(body: content, mimeType: 'text/html',
             replyTo: '$DEFAULT_REPLYTO', subject: subject,
             to: to, attachLog: true )
        }
    }
    //send slack notif
    slackSend (color: '#FF0000', message: "The [${env.JOB_NAME}#${currentBuild.number}](${currentBuild.absoluteUrl}) job finished with status ${currentBuild.currentResult}", channel: "#pd-jenkins"
}
}
```

37

Pipeline - Email Integration

Usefull to alert developers of failed builds

Increase of produtivity (faster bug detection)



38

Pipeline – Slack Integration

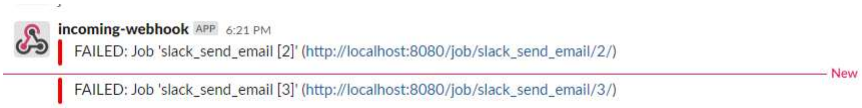
Slack is a chat tool.

Very usefull to integrate several platforms into Slack chats.

- Failed builds
- Production alarms.
- …

39

Pipeline – Slack Integration



40

Some bibliography

- https://www.jenkins.io/
- https://medium.com/swlh/jenkins-pipeline-to-create-docker-image-and-push-to-docker-hub-721919512f2
- http://jenkinsci.github.io/job-dsl-plugin/
- http://jenkinsci.github.io/job-dsl-plugin/#method/javaposse.jobdsl.dsl.helpers.step.StepContext.dockerBuildAndPublish
- https://plugins.jenkins.io/groovy-postbuild

41